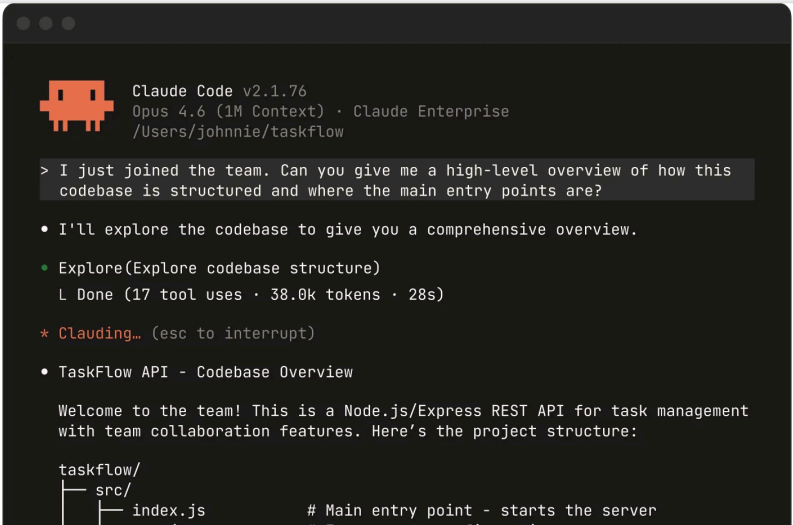


에이전틱 코딩

(Agentic Coding - Claude Code)



```
Claude Code v2.1.76
Opus 4.6 (1M Context) · Claude Enterprise
/Users/johnnie/taskflow

> I just joined the team. Can you give me a high-level overview of how this
  codebase is structured and where the main entry points are?

• I'll explore the codebase to give you a comprehensive overview.
  • Explore(Explore codebase structure)
    L Done (17 tool uses · 38.0k tokens · 28s)
  * Clauding... (esc to interrupt)

• TaskFlow API - Codebase Overview

Welcome to the team! This is a Node.js/Express REST API for task management
with team collaboration features. Here's the project structure:

taskflow/
├── src/
│   └── index.js      # Main entry point - starts the server
```

왜 "에이전틱 코딩"인가

Vibe Coding(Software 3.0)에서 Agentic Coding(LLM OS)으로 — 대화창(Chat)에서 작업장(Workspace)으로

"바이브 코딩은 **대화창(Chat)**에서 시작한 일을 끝내는 과정에 머문다.
에이전틱 코딩은 **작업장(Workspace)** 위에서 실제 시스템으로 이어진다."

— Andrej Karpathy: "Software 3.0 → LLM OS" · Anthropic: 2026 Agentic Coding Trends Report

BEFORE · VIBE CODING

대화창(Chat)에서 머물다

- ✗ 요청 한 번에 **답 한 번**의 왕복
- ✗ 보안 취약점 ~45% · 출력 불일치
- ✗ 운영·감사·복구 메커니즘 **부재**
- ✗ METR 보고서: 과제 길이 길수록 **성공률 급락**

NOW · AGENTIC CODING

작업장(Workspace)으로 넘어가다

- ✓ 위임·지시·판별·검증의 사이클
- ✓ 도구·메모리·컨텍스트·제약이 **결합**
- ✓ Claude Code · Skills · Plugins · MCP 생태계
- ✓ Anthropic 2026 리포트: "**learning-by-doing**"

핵심: "AI가 너무 똑똑해서가 아니라, 작업이 단순 답변보다 복잡해서" 에이전틱 코딩이 필요하다

핵심 공식 — Agent = Model + Harness

날것의 LLM은 에이전트가 아니다 — 하네스가 모델을 감싸야 비로소 에이전트가 된다

Agent = Model + Harness

모델은 "지능"일 뿐 — 도구·메모리·컨텍스트·검증·제약이 결합되어야 에이전트가 된다

MODEL

지능 (Intelligence)

LLM의 **추론·생성 능력** 자체. 모델은 점점 범용품(commodity)이 되어가고 있다 — Sonnet, GPT, Gemini 모두 비슷한 수준에 수렴. **차별화 요소가 모델 쪽이 아니라 하네스 쪽으로 이동**하는 중이다.

HARNESS

제어 환경 (Control Environment)

모델 주위의 **모든 장치** — 도구, 메모리, 컨텍스트, 검증, 제약, 복구 메커니즘. 말(馬)에 채우는 **고삐와 안장**의 비유.

"에이전트가 실수할 때마다, 그 실수를 두 번 다시 일으키지 않게 하는 장치를 설계한다."

— Mitchell Hashimoto (2026.2.5)

에이전틱 코딩 = 모델을 어떻게 쓰느냐가 아니라 하네스를 어떻게 설계하느냐의 문제

4가지 작업 경로 — 책상을 고르면 결과가 바뀐다

Chat · Projects · Cowork · Claude Code — 같은 Claude라도 어디에 앉느냐에 따라 답이 달라진다

CHAT

01

Chat



질문하고 답을 받는 곳 — **상담 창구**. 빠른 사고 보조, 아이디어 발산.

일회성 Q&A · 요약 · 분류 · 번역

PROJECTS

02

Projects



같은 배경 자료를 계속 참고하는 곳 — **공용 배경 자료함**. 반복 업무의 기본 환경.

문서 템플릿 작업 · 반복 분류 · 회사 지침 기반 작성

COWORK

03

Cowork



자료를 펼쳐 두고 조연자와 함께 일하는 곳 — **실무 책상**. 문서·결과물 오가는 협업.

기획 리서치 · 보고서 작성 · 콘텐츠 생산 팀

CLAUDE CODE

04

Claude Code



공구와 검수표가 함께 놓인 **개발 작업대**. 코드·테스트·리뷰·배포까지 연결된 시스템.

코딩 에이전트 · 복수 파일 수정 · 테스트·PR 자동화

 사무실에 비유하면 4개의 **다른 책상**이다 — 제품이 따로 있는 게 아니라, 같은 동료를 **어떤 책상에 앉혔는지에** 따라 결과가 달라진다.

대화창(Chat) → 작업장(Workspace)으로의 전환: **Chat → Projects → Cowork → Claude Code** 순으로 하네스가 짙어진다

Claude Code 렌즈 — 무엇이 다른가

Claude Code를 "한 줄 공식"으로 보면 — 기억·도구·권한·세션이 결합된 실행 시스템

Claude Code = memoized context + tool orchestration + permission gate + resumable session

01 / LENS

Memoized Context



한 번 정한 배경을 **반복 재사용**.
CLAUDE.md · 지침 · 대화 이력의 누적.

02 / LENS

Tool Orchestration



여러 도구를 **순서와 조합**으로 조율. 검색·편집·테스트·커밋 연결.

03 / LENS

Permission Gate



실행 전에 **허용·질문·차단**을 가르는 관문. 위험 작업은 승인 필수.

04 / LENS

Resumable Session



세션이 **끊겨도 상태를 이어보기**. 장기 실행 워크플로의 핵심.

05 / SURFACE

Multi-Surface



CLI · API · MCP · VM — 하나의 에이전트가 **여러 표면**에서 동작.

CLI · 터미널 기반 실행

API · 외부 시스템 연결

MCP · 외부 도구/서비스 연결 규약

VM · 격리된 분리 실행 환경

Claude Code는 "단순 실행 툴"이 아니라 **작업 구조를 감싸주는 하네스**로 봐야 한다

Claude Code 4가지 환경 — 어디서 실행하나

같은 엔진, 같은 CLAUDE.md · 설정 · MCP — 환경은 다르지만 작업은 이어진다



01 · CLI

Terminal

기본 진입점. **모든 기능** 지원. 스크립트·파이프·CI에 최적.

`$ claude`



02 · IDE

VS Code

인라인 **diff**·@mention·계획 검토·대화 기록을 편집기에서.

`Cmd/Ctrl + Shift + P`



03 · APP

Desktop

시각적 diff 검토 · **여러 세션 병렬** · 스케줄링 · 클라우드 세션.

`macOS / Windows`



04 · WEB

Web

설치 불필요. **긴 작업**을 시작하고 나중에 확인. iOS 앱 연동.

`claude.ai/code`

INSTALL

macOS/Linux/WSL `curl -fsSL https://claude.ai/install.sh | bash`

Homebrew `brew install --cask claude-code`

Windows `irm https://claude.ai/install.ps1 | iex`

핵심: 환경이 달라도 CLAUDE.md · 설정 · MCP 서버는 공통 — 책상을 옮겨도 작업이 이어진다 · 출처: code.claude.com/docs/ko/

Claude Code — 9가지 공식 활용 시나리오

Anthropic 공식 문서 기준 — 단일 개발자를 넘어 팀·CI·채팅·모바일까지

01

□ 지루한 작업 자동화

테스트 작성, lint 수정, **merge conflict** 해결, 의존성 업데이트, 릴리스 노트.

02

🔧 기능 구축 · 버그 수정

자연어로 설명 → Claude Code가 **계획·구현·검증**까지. 오류 메시지 기반 근본 원인 추적.

03

🌿 Git · PR 생성

변경사항 스테이징 → 커밋 메시지 → 브랜치 → **PR 오픈**까지 한 번에.

04

🔗 MCP로 도구 연결

Google Drive 설계 문서, Jira 티켓, Slack 데이터 — **커스텀 MCP**로 확장.

05

📦 Skills · Hooks 커스터마이징

CLAUDE.md + **Skills** + **Hooks**로 팀 워크플로우 패키징. `/review-pr`, `/deploy-staging`.

06

👥 에이전트 팀 · SubAgents

여러 Claude Code 에이전트 병렬 실행 + **Agent SDK**로 완전 커스텀.

07

📄 CLI 파이프 · 스크립트

`tail app.log | claude -p "..."` — Unix 철학에 따른 **파이프 조합**.

08

🕒 예약·반복 실행

클라우드/데스크톱 **스케줄 태스크**. 야간 CI 분석, 주간 의존성 감사.

09

🌐 어디서나 · 환경 간 이동

원격 제어, Dispatch, **/teleport**, `/desktop` — 환경 간 작업 이동.

INTEGRATIONS · 바로 쓸 수 있는 공식 통합

GitHub Actions · PR 리뷰 자동화

GitLab CI/CD · 파이프라인 연동

Slack · @Claude 멘션 → PR

Chrome · 라이브 웹 디버깅

GitHub Code Review · 자동 코드 검토

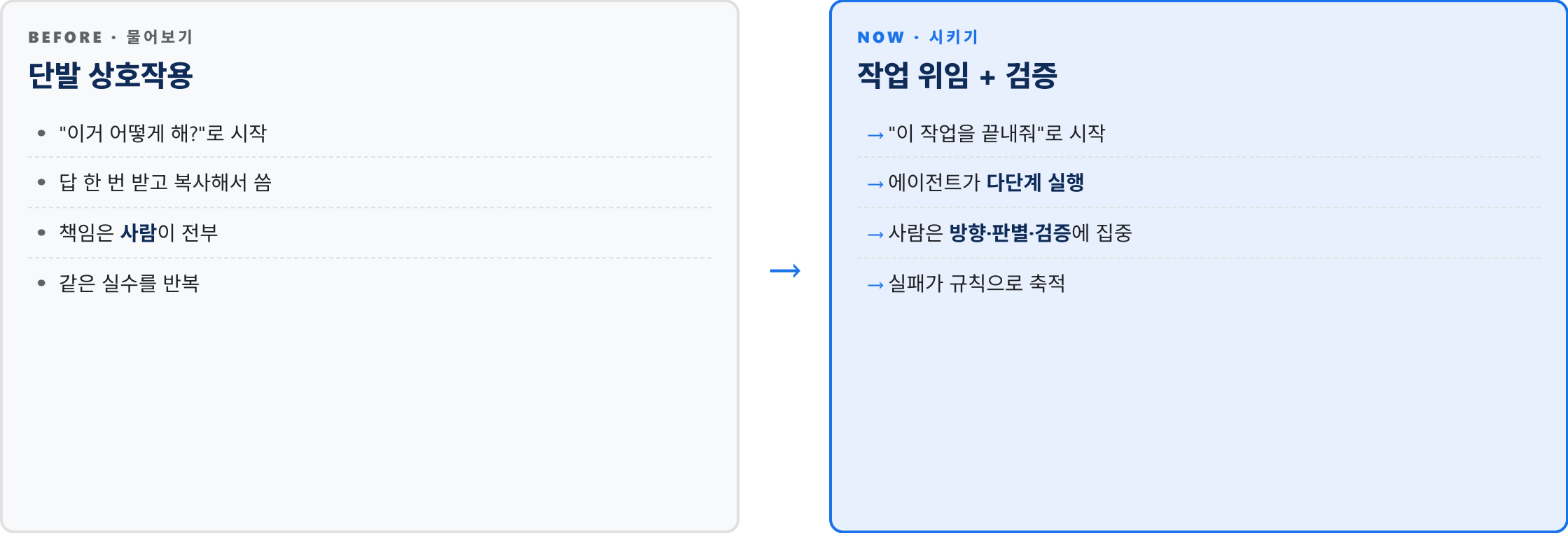
Channels · Telegram · Discord · 웹훅

Remote Control · 휴대폰 제어

iOS 앱 · 모바일 세션 시작

물어보기 vs 시키기 — 4D 프레임워크

2026년의 AI 사용자를 가르치는 축은 "물어보는가"가 아니라 "어떻게 시키는가"



CLAUDE.md · AGENTS.md — 에이전트의 학습된 지혜

프로젝트 루트에 두는 마크다운 한 장 — 하네스 엔지니어링의 출발점

실패 1건 = 방지책 1줄

삭제하지 않고 누적한다 — Hashimoto 원칙

운영 팁

- ▶ "~하지 마라"만 쓰지 말고 "대신 ~해라"까지
- ▶ 자동 생성 금지 — ETH 연구: 성능 3%↓, 비용 20%↑
- ▶ 한 달 6~8줄, 1년이면 50줄 넘는 **학습된 지혜**
- ▶ 새로운 에이전트·팀원의 **운영 매뉴얼** 역할

CLAUDE.md (프로젝트 루트)

프로젝트 개요

- 이름: payment-service
- 런타임: Python 3.12 / uv

빌드 & 실행

- 의존성: `uv sync`
- 테스트: `uv run pytest -q`

코드 컨벤션

- 포맷터: ruff format
- 타입 검사: mypy --strict src/

에이전트 주의사항

- `secrets/`, `.env\*` 절대 읽거나 출력 금지
- DB 마이그레이션은 사람 검토 후 적용
- 외부 API 호출 전 **--dry-run** 검증 필수

Past Failures → Prevention

- # 2026-02-15: 메모리 누수 발견
 - std.mem.alloc 직접 호출 금지 → arena 사용
- # 2026-03-02: 롤백 실패
 - 마이그레이션은 **reversible** 필수

하네스 4기둥 — Constrain · Inform · Verify · Correct

독립된 4개 영역이 아니라 순환하는 사이클 — 돌수록 하네스는 똑똑해진다

CONSTRAIN · 제약

Constrain

"무엇을 못 하게 할 것인가"

- 샌드박스 격리
- 도구 허용 목록
- 비용·호출 상한
- 파일 경로 화이트리스트

01

INFORM · 정보 제공

Inform

"무엇을 해야 하는지 어떻게 알릴까"

- AGENTS.md / CLAUDE.md
- MCP 동적 컨텍스트
- 시스템 프롬프트
- 이벤트 리마인더

02

VERIFY · 검증

Verify

"제대로 했는지 어떻게 확인할까"

- Eval 파이프라인
- 자동 테스트
- LLM-as-Judge
- 린터·보안 스캐너

03

CORRECT · 수정

Correct

"실수했을 때 어떻게 복구할까"

- 재시도 · 백오프
- Human-in-the-Loop
- 체크포인트 롤백
- 가비지 컬렉션

04

CYCLE

Correct에서 발견된 패턴이 **Constrain의 새 규칙**이 되고, 그게 **Inform을 강화**하고, 새로운 **Verify**가 추가된다 — 한 바퀴 돌 때마다 하네스는 한 단계 진화한다

4기둥을 체크리스트로: "이 작업에 제약·정보·검증·복구가 모두 준비됐나?"

Skills — 재사용 능력 모듈

에이전트에게 "이런 일은 이렇게 처리해"를 가르치는 표준 단위 — 필요할 때 동적으로 로드

Skills란?

특정 작업 영역에 대한 **지침·예시·도구**를 하나의 폴더로 묶은 것. 에이전트가 관련 작업을 만나면 자동으로 로드되어 **전문성을 빌려 쓴다**.

핵심 특징

- ✓ **동적 로딩** — 필요할 때만 컨텍스트에 주입
- ✓ **모듈화** — 팀 전체가 공유·재사용
- ✓ **축적 가능** — 한 번 만든 Skill은 영구 자산
- ✓ **공식·커뮤니티** Skill 생태계 존재

```
# .claude/skills/pr-review/SKILL.md
```

```
---
```

```
name: pr-review
```

```
description: "PR 리뷰 체크리스트 수행"
```

```
when_to_use: "PR 리뷰 요청 시"
```

```
---
```

```
# PR Review Skill
```

```
## 체크 순서
```

```
1. 변경된 파일 전체 읽기
```

전략적 활용

- ✓ 반복 작업 = Skill로 추출
- ✓ 회사 고유 **플레이북**을 Skill로 인코딩
- ✓ 신규 입사자 온보딩 자동화



Writing Skill



Testing Skill



Security Skill



Data Skill

Skills = 팀의 암묵지를 AI가 읽을 수 있는 형태로 기록한 것

Plugins · MCP · Hooks — 확장과 자동화

에이전트를 외부 세계와 연결하는 3가지 방식 — 각각 용도가 다르다

INTERNAL EXTENSION

Plugins

Claude Code 내부 확장

Claude Code 자체의 기능을 **슬래시 명령어·자동화**로 확장. 팀 전용 커스텀 동작을 추가.

▶ /commit, /review 같은 커스텀 슬래시

▶ 팀 전용 워크플로우 버튼화

▶ 로컬에서 즉시 배포 가능

EXTERNAL PROTOCOL

MCP

Model Context Protocol

외부 시스템과 연결하는 **표준 규약**. 한 번 구현하면 **여러 에이전트**가 공유 가능.

▶ DB · 사내 API · Slack · GitHub 연결

▶ USB 표준 같은 범용 인터페이스

▶ 공식·커뮤니티 서버 생태계

EVENT LISTENER

Hooks

이벤트 기반 자동 실행

특정 이벤트(도구 호출 전·후, 세션 시작 등)에 **자동 실행**되는 **훅**. 정책 강제·로깅·알림.

▶ pre-tool · post-tool 훅

▶ 위험 작업 자동 차단·알림

▶ 감사 로그 자동 기록

선택 기준

내부 단축키는 **Plugins** · 외부 시스템 통합은 **MCP** · 정책·감사 강제는 **Hooks** — 혼동하지 말고 용도에 맞게 조합

Skills + Plugins + MCP + Hooks = **에이전트 생태계 확장의 4대 축**

Verify — Eval 주도 개발(EDD)

AI 출력은 확률적이다 → 검증도 **확률적 임계값**으로 설계해야 한다

TDD · 전통 개발

결정론적 단언

```
assert output == expected
```

EDD · AI 시대

확률적 임계값

```
assert success_rate >= 0.8
```

Anthropic 권장 임계값

🔒 보안 관련	≥ 95%
👉 코드 포맷·스타일	≥ 90%
⚙️ 기능 구현	70~80%
💡 창의적 작업	50~70%

```
# eval.py
def run_eval(task, n=10, threshold=0.8):
    results = []
    for _ in range(n):
        out = agent.invoke(task)
        results.append(grader(out))

    success_rate = sum(results) / n
    assert success_rate >= threshold, \
        f"실패: {success_rate:.1%}"
    return success_rate

# 사용
run_eval(
    "로그인 버그 수정",
    n=10,
    threshold=0.85
```

검증의 3가지 유형

- ▶ **코드 실행** — 컴파일·테스트 통과 여부 (객관)
- ▶ **셀프 검증** — LLM-as-Judge로 다른 LLM이 평가
- ▶ **외부 도구** — 린터·시큐리티 스캐너 등

"1번 정답"이 아니라 **"N번 중 M번 이상 성공"** — AI 시대 검증의 새 패러다임

Best Practices — 탐색 · 계획 · 구현 · 커밋 4단계

Anthropic 내부 팀이 검증한 워크플로우 · Plan Mode로 잘못된 문제를 푸는 코드 방지



검증 기준 제공

테스트 케이스·스크린샷·예상 출력을 주면 Claude가 스스로 확인한다
— 최고 레버리지

구체적 컨텍스트

@file.ts로 파일 참조, 기존 패턴 지적, 증상+위치+"수정된 모습" 포함

Plan Mode 건너뛰기

diff를 한 문장으로 설명 가능한 작업(오타·rename·로그 한 줄)은 계획 생략

세션 · 컨텍스트 관리 — 가장 중요한 리소스

Context window가 채워질수록 성능은 떨어진다 · 관리해야 할 #1 자원

FIRST PRINCIPLE

Context window는 관리해야 할 가장 중요한 자원이다

대화·파일·명령 출력이 모두 쌓인다. 가득 차면 Claude는 지시를 "잊고" 실수가 늘어난다. **/clear**를 자주 쓰고, **subagent**로 조사를 위임해 메인 컨텍스트를 깨끗하게 유지하라.

SESSION CONTROL

--continue 최근 대화 이어서

claude --continue · 작업이 여러 세션에 걸쳐도 **컨텍스트 재설명 불필요**

PARALLEL WORK

--worktree Git worktree 격리

claude -w feature-auth · 여러 세션이 **서로 충돌 없이** 병렬 작업

CONTEXT RESET

/clear 컨텍스트 완전 초기화

관련 없는 작업 간 전환 시 필수. **두 번 이상 수정 실패**했다면 /clear 후 새 프롬프트

CHECKPOINTS

Esc Esc **/rewind** — 체크포인트 복원

변경 전 자동 체크포인트. 대화·코드 선택 복원. **위험한 시도도 되돌릴 수 있다**

COMPACT

/compact 대화 요약·압축

/compact Focus on API changes · 중요 정보 보존하며 공간 확보

SIDE QUESTIONS

/btw 대화 기록 없이 질문

답변이 오버레이로만 표시 · **컨텍스트 오염 없음**

DELEGATE

subagents 조사·검증 위임

"use subagents to investigate X" · **별도 컨텍스트에서 탐색** 후 요약만 보고

피해야 할 5가지 안티패턴 — 흔한 실패 회피 가이드

Anthropic 공식 문서 기준 · 조기에 인식하면 수 시간을 아낀다

AP 01



주방 싱크 세션

한 작업 → 관련 없는 질문 → 다시 원래 작업. **컨텍스트가 잡동사니**로 가득.

→ 수정 작업 간 /clear

AP 02



반복 수정 루프

실수 → 수정 → 여전히 실수 → 수정. **실**
패 접근법으로 컨텍스트 오염.

→ 수정 2회 실패 후 /clear + 더 나은 초기 프롬프트

AP 03

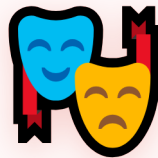


과다 지정 CLAUDE.md

너무 길어서 Claude가 절반을 무시. **중**
요 규칙이 노이즈에 묻힘.

→ 수정 무자비하게 정리 · hooks로 변환

AP 04



신뢰-검증 간격

그렇듯해 보이지만 **옛지 케이스**를 처리
못하는 코드가 생성됨.

→ 수정 테스트·스크립트·스크린샷으로 검증 필수

AP 05



무한 탐색

범위 없이 "조사"하면 수백 파일을 읽으며 **컨텍스트 고갈**.

→ 수정 조사 범위 좁히기 · subagent에 위임

KEY TAKEAWAY

누적된 수정이 있는 긴 세션보다 **배운 내용을 통합한 깨끗한 새 세션**이 거의 항상 더 나은 성능을 낸다. 궤도 이탈이 보이면 즉시 **Esc** 또는 **/clear**.

직무별 플레이북 — 누가 어디에 앉을까

같은 Claude라도 직무에 따라 주 작업 경로가 다르다 — 먼저 책상을 고르고 하네스를 설계하라



개발자

주: CLAUDE CODE

> 다파일 수정 · 테스트 자동화

> PR 리뷰 Skill로 품질 게이트

> MCP로 DB·Jira·GitHub 연결

> CLAUDE.md에 컨벤션 고정



기획자 · PM

주: COWORK + PROJECTS

> 기획서 초안 + 시장 리서치

> 회의록·액션 아이템 자동 정리

> 의사결정 로그 누적 관리

> Projects에 사내 템플릿 보관



데이터 분석가

주: CLAUDE CODE + COWORK

> SQL·Python 분석 파이프라인

> 대시보드 자동 생성

> Notebook 에이전트로 EDA

> 보고서 초안 Cowork에서



디자이너 · 마케터

주: COWORK + CHAT

> 카피·크리에이티브 발산

> 브랜드 가이드 기반 반복 생성

> 멀티모달(이미지+텍스트)

> 캠페인 기획 협업

한국 실무 팁

"한국어 고유 예절/문체"는 CLAUDE.md에 명시 · 보안 민감한 사내 데이터는 MCP로 분리 · 규제 영역(금융·의료)은 Permission Gate 엄격화

"내 직무에 맞는 책상과 하네스는 무엇인가?"부터 답하라 — **툴을 먼저 고르는 실수 금지**

보안 · 거버넌스 — Permission Gate 3단 설계

에이전트 자율성 ↑ = 위험 ↑ → 실행 전에 허용·질문·차단의 3단 관문이 필요

TIER 1 · AUTO-ALLOW 자동 허용 사람 승인 불필요 — 자유롭게 ✓ 읽기 작업 (파일·DB·API GET) ✓ 테스트 실행 (격리된 환경) ✓ Lint·포맷 자동화 ✓ 로그·메트릭 수집	TIER 2 · ASK 사람에게 질문 작업 전 승인 1회 — Y/N ? 파일 생성·수정 (로컬) ? 외부 API 호출 (비용 발생) ? git commit / PR 생성 ? 의존성 추가·변경	TIER 3 · DENY 자동 차단 사람이 직접 수행 — 에이전트 불가 ✗ .env · secrets/ 읽기·출력 ✗ 프로덕션 DB 직접 쓰기 ✗ git push --force · branch 삭제 ✗ 결제·외부 메시지 전송
---	---	--

3 원칙	① 최소 권한 — 필요 없는 건 처음부터 차단	② 감사 가능성 — 모든 행동에 로그	③ 복구 가능성 — 실수 뒤로 돌릴 수 있게
------	---------------------------	----------------------	--------------------------

"에이전트는 신입 인턴처럼 대하라" — 신뢰는 권한 축적으로 증명된다

월요일 아침 — 3가지 액션으로 시작

거창한 인프라가 아니다 — 프로젝트 루트의 마크다운 파일 한 장에서 시작한다

1

ACTION · TODAY

CLAUDE.md 만들기

프로젝트 루트에 **CLAUDE.md** 한 장을 만든다. 빌드·테스트 명령, 코드 컨벤션, 에이전트 주의사항만 있으면 충분. **완벽할 필요 없다.**

5분이면 충분 · 시작이 중요

2

PRINCIPLE · DAILY

실수 1건 = 규칙 1줄

에이전트가 실수할 때마다 **AGENTS.md**에 **방지 규칙 한 줄** 누적. 한 달이면 6~8줄, 1년이면 50줄 이상 — 팀의 **학습된 지혜**가 된다.

삭제 금지 · 오직 추가

3

MEASURE · WEEKLY

3가지 숫자 추적

테스트 통과율 · 토큰 비용 · 작업 시간 — 주간 단위로 이 셋만 봐도 하네스의 건강 상태가 보인다. "숫자가 진실"이다.

스프레드시트 한 장이면 충분

추적 지표 예시 (베이스라인)

≥ 85%

테스트 통과율 (Eval)

-30%

주간 토큰 비용 추이

2x

작업 소요 시간 단축

+50

연간 AGENTS.md 규칙

"중요한 일을 빨리 끝낸다"보다 "작고 되돌리기 쉬운 일을 먼저 맡긴다"가 더 중요

에이전틱 코딩의 본질

The real game is harness, not the model

"모델은 범용품이 되어간다.
오늘 우리의 경쟁력은 하네스에 있다."

— 같은 Claude라도, 어떤 책상과 어떤 규칙 위에 앉히느냐가 결과를 가른다

TAKEAWAY 1

Agent = Model + Harness

LLM만으로는 부족하다 — 도구·컨텍스트·검증·제약이 결합되어야 에이전트

TAKEAWAY 2

대화창 → 작업장

물어보는 도구에서 시키는 도구로 — 위임·지시·판별·검증의 사이클

TAKEAWAY 3

실패 1건 = 규칙 1줄

CLAUDE.md에 누적되는 학습된 지혜가 팀의 진짜 자산

THANK YOU